

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)» (МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ КАФЕДРА
СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

РУБЕЖНЫЙ КОНТРОЛЬ 1

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-31Б _____ (Подпись) Минский И.Р. (Ф.И.О.)

Проверил: _____ (Подпись) Гапанюк Ю.Е. (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026

Вариант А16.

Предметная область:

Книга – Книжный магазин

Постановка задачи:

Разработать программу на языке Python: создать классы «Книга» и «Книжный магазин» (связь один-ко-многим), класс «Книги магазина» (связь многие-ко многим), заполнить списки тестовыми данными и реализовать три запроса варианта.

А1. Вывести список всех связанных книг и магазинов, отсортированный по магазинам (один-ко-многим).

А2. Вывести список магазинов с суммарной стоимостью книг в каждом, отсортированный по суммарной стоимости (один-ко-многим). Для этого в класс «Книга» добавлен количественный признак «цена».

А3. Вывести список всех магазинов, в названии которых присутствует слово «магазин», и список книг в них (многие-ко-многим).

Исходный код программы:

```
from dataclasses import dataclass
from operator import itemgetter


@dataclass
class Book:
    """Книга (сторона «много» связи один-ко-многим)."""
    id: int
    title: str
    price: float       # количественный признак
    store_id: int      # ID основного книжного магазина


@dataclass
class BookStore:
    """Книжный магазин."""
    id: int
    name: str
```

```
@dataclass
class StoreBook:
    """Книги магазина (связь многие-ко-многим)."""
    book_id: int
    store_id: int

# Тестовые данные
stores = [
    BookStore(1, "Книжный магазин «Читай-город»"),
    BookStore(2, "Дом книги"),
    BookStore(3, "Книжный магазин «Москва»"),
    BookStore(4, "Букинист"),
]

books = [
    Book(1, "Война и мир", 1200.0, 1),
    Book(2, "Мастер и Маргарита", 650.0, 1),
    Book(3, "Python. К вершинам мастерства", 2300.0, 2),
    Book(4, "Чистый код", 1800.0, 2),
    Book(5, "Преступление и наказание", 500.0, 3),
    Book(6, "Евгений Онегин", 350.0, 4),
]

stores_books = [
    StoreBook(1, 1), StoreBook(2, 1), StoreBook(5, 1),
    StoreBook(3, 2), StoreBook(4, 2), StoreBook(1, 2),
    StoreBook(5, 3), StoreBook(2, 3), StoreBook(4, 3),
    StoreBook(6, 4), StoreBook(1, 4),
]

def main():
    # Соединение данных один-ко-многим
```

```
    one_to_many = [(b.title, b.price, s.name)
                   for s in stores
                   for b in books
                   if b.store_id == s.id]

    # Соединение данных многие-ко-многим
    many_to_many_temp = [(s.name, sb.store_id, sb.book_id)
                         for s in stores
                         for sb in stores_books
                         if s.id == sb.store_id]
    many_to_many = [(b.title, b.price, store_name)
                    for store_name, _, book_id in
many_to_many_temp
                    for b in books if b.id == book_id]

    print("Задание A1")
    res_1 = sorted(one_to_many, key=itemgetter(2))
    for row in res_1:
        print(row)

    print("\nЗадание A2")
    res_2 = sorted(
        ((s.name, sum(price for _, price, name in one_to_many
                      if name == s.name))
         for s in stores),
        key=itemgetter(1), reverse=True)
    for row in res_2:
        print(row)

    print("\nЗадание A3")
    res_3 = {s.name: [title for title, _, name in many_to_many
                      if name == s.name]
             for s in filter(lambda s: "магазин" in
s.name.lower(), stores)}
```

```
    for name, titles in res_3.items():
        print(f"{name!r}: {titles}")

if __name__ == "__main__":
    main()
```

Результаты выполнения программы

